

# PRACTICAL WEEK - 8

TASK 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
siddu@DESKTOP-5CRM05P:~$ mkdir -p ~/practical_memory
cd ~/practical_memory
siddu@DESKTOP-5CRM05P:~/practical_memory$ nano task1.c
siddu@DESKTOP-5CRM05P:~/practical_memory$ gcc -g -Wall task1.c -o task1
siddu@DESKTOP-5CRM05P:~/practical_memory$ ./task1
malloc array: 1 2 3 4 5
calloc array (initial zeros): 0 0 0 0 0
realloc array: 1 2 3 4 5 6 7 8 9 10
Leaked block address: 0x5d62165d7790, leak[0] = 42
siddu@DESKTOP-5CRM05P:~/practical_memory$ valgrind --leak-check=full --show-leak-kinds=all ./task1
==639== Memcheck, a memory error detector
==639== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==639== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==639== Command: ./task1
==639==
malloc array: 1 2 3 4 5
calloc array (initial zeros): 0 0 0 0 0
realloc array: 1 2 3 4 5 6 7 8 9 10
Leaked block address: 0x4a915b0, leak[0] = 42
==639==
==639== HEAP SUMMARY:
==639==   in use at exit: 400 bytes in 1 blocks
==639==   total heap usage: 5 allocs, 4 frees, 1,504 bytes allocated
==639==
==639== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==639==   at 0x4850858: malloc (vg_replace_malloc.c:447)
==639==   by 0x400147D: main (task1.c:63)
==639==
==639== LEAK SUMMARY:
==639==   definitely lost: 400 bytes in 1 blocks
==639==   indirectly lost: 0 bytes in 0 blocks
==639==   possibly lost: 0 bytes in 0 blocks
==639==   still reachable: 0 bytes in 0 blocks
==639==   suppressed: 0 bytes in 0 blocks
==639== For lists of detected and suppressed errors, rerun with: -s
==639== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
siddu@DESKTOP-5CRM05P:~/practical_memory$
```

```
GNU nano 8.7.1 task1.c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    // malloc()
    int *arr1 = (int *)malloc(5 * sizeof(int));
    if (arr1 == NULL) {
        perror("malloc failed");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr1[i] = i + 1;
    }

    printf("malloc array: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr1[i]);
    }
    printf("\n");

    // calloc()
    int *arr2 = (int *)calloc(5, sizeof(int));
    if (arr2 == NULL) {
        perror("calloc failed");
        free(arr1);
        return 1;
    }

    printf("calloc array (initial zeros): ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr2[i]);
    }
    printf("\n");

    // realloc()
    int *temp = (int *)realloc(arr1, 10 * sizeof(int));
    if (temp == NULL) {
        perror("realloc failed");
        free(arr1);
        free(arr2);
        return 1;
    }
}
```

```
GNU nano 8.7.1 task1.c
// realloc()
int *temp = (int *)realloc(arr1, 10 * sizeof(int));
if (temp == NULL) {
    perror("realloc failed");
    free(arr1);
    free(arr2);
    return 1;
}
arr1 = temp;

for (int i = 5; i < 10; i++) {
    arr1[i] = i + 1;
}

printf("realloc array: ");
for (int i = 0; i < 10; i++) {
    printf("%d ", arr1[i]);
}
printf("\n");

// free()
free(arr1);
free(arr2);
arr1 = NULL;
arr2 = NULL;

// Intentional leak for Valgrind demonstration
int *leak = (int *)malloc(100 * sizeof(int));
if (leak == NULL) {
    perror("malloc leak failed");
    return 1;
}

leak[0] = 42;
printf("Leaked block address: %p, leak[0] = %d\n",
      (void *)leak, leak[0]);

// Do NOT free(leak) -- this creates a memory leak on purpose
// so Valgrind can detect it.

return 0;
}
```

TASK 2: Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
siddu@DESKTOP-5CRM05P: ~$ mkdir -p ~/practical_memory
cd ~/practical_memory
siddu@DESKTOP-5CRM05P:~/practical_memory$ nano copy_on_write.c
siddu@DESKTOP-5CRM05P:~/practical_memory$ gcc -Wall copy_on_write.c -o copy_on_write
siddu@DESKTOP-5CRM05P:~/practical_memory$ ./copy_on_write
Parent PID: 852
Memory allocated and initialized.
Address of data: 0x6eaf8Cdb010
Creating child process using fork(...)

Parent Process:
Parent PID: 852
Child PID: 853
Parent data[0] = 0

Child Process:
Child PID: 853
Address of data in child: 0x6eaf8Cdb010
Before modification: data[0] = 0
After modification: data[0] = 9999
Child modified the memory.
Copy-on-write creates a private copy of the modified page.
After child modification, parent data[0] = 0
Parent still has its original data.
```

```

GNU nano 8.7.1                                copy_on_write.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void) {
    int *data = (int *)malloc(100 * sizeof(int));
    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    data[0] = 0;

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated and initialized.\n");
    printf("Address of data: %p\n", (void *)data);
    printf("Creating child process using fork(...)\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Address of data in child: %p\n", (void *)data);
        printf("Before modification: data[0] = %d\n", data[0]);

        // Modify the data
        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);
        printf("Child modified the memory.\n");
        printf("Copy-on-write creates a private copy of the modified page.\n");
    }
}

```

```

GNU nano 8.7.1                                copy_on_write.c
        perror("fork");
        free(data);
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Address of data in child: %p\n", (void *)data);
        printf("Before modification: data[0] = %d\n", data[0]);

        // Modify the data
        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);
        printf("Child modified the memory.\n");
        printf("Copy-on-write creates a private copy of the modified page.\n");

        sleep(5);
        free(data);
        exit(0);
    }
    else {
        // Parent process
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent data[0] = %d\n", data[0]);

        sleep(2);

        printf("After child modification, parent data[0] = %d\n", data[0]);
        printf("Parent still has its original data.\n");

        wait(NULL);
        free(data);
    }

    return 0;
}

```